

DATA SCIENCE KNOWLEDGE BASE

A concise reference for core Data Science concepts, compiled for the DataSense AI assistant.

Demonstration Knowledge Base — for educational and demonstration purposes only. This document is not official documentation and is used to illustrate a Retrieval-Augmented Generation (RAG) pipeline.

Chapter 1: Data Cleaning

Data cleaning is the process of detecting and correcting inaccurate, incomplete, or inconsistent records within a dataset before it is used for analysis or modeling.

Common data cleaning tasks include handling missing values, removing duplicate records, fixing inconsistent formatting, correcting data types, and standardizing categorical labels.

Clean data is a prerequisite for reliable analysis. Skipping this step often leads to misleading statistics and poorly performing models.

- Detect and remove duplicate rows.
- Standardize inconsistent categories (e.g. 'Male', 'male', 'M').
- Correct data types (e.g. numeric columns stored as text).
- Validate value ranges against domain knowledge.

Chapter 2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis is the practice of summarizing a dataset's main characteristics, often using descriptive statistics and visualizations, before formal modeling begins.

EDA helps analysts understand data distributions, detect anomalies, identify relationships between variables, and form hypotheses that guide further analysis.

- Inspect dataset shape, column types, and basic statistics.
- Visualize distributions (histograms, box plots) and relationships (scatter plots, correlation matrices).
- Identify data quality issues early: missing values, duplicates, outliers.

Chapter 3: Missing Values

Missing values occur when no data value is stored for a variable in an observation. They can arise from data entry errors, system limitations, or genuinely unavailable information.

Common strategies for handling missing values include removal (dropping rows or columns), imputation (mean, median, mode, or model-based), and flagging (creating an indicator column).

The right strategy depends on the proportion of missing data, whether it is missing at random, and the importance of the affected column to the analysis.

- Drop rows/columns when missingness is minimal and non-informative.

- Impute with mean/median for numeric columns, mode for categorical columns.
- Consider model-based imputation for more complex patterns.
- Always document how missing values were handled.

Chapter 4: Outliers

An outlier is an observation that deviates significantly from the rest of the dataset. Outliers can result from measurement error, data entry mistakes, or genuine rare events.

A common statistical method for detecting outliers is the Interquartile Range (IQR) method: values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are flagged as potential outliers.

Outliers are not always errors — some represent meaningful rare events and should be investigated rather than removed automatically.

- Use box plots or the IQR method to flag potential outliers.
- Investigate the business context before removing an outlier.
- Consider robust statistics (median, IQR) when outliers are present.

Chapter 5: Feature Engineering

Feature engineering is the process of using domain knowledge to create new input variables (features) that make machine learning models more effective.

Examples include extracting date parts (day, month, weekday) from a timestamp, encoding categorical variables, scaling numeric features, and combining existing columns into ratios.

Good feature engineering often has a larger impact on model performance than the choice of algorithm.

- Encode categorical variables (one-hot encoding, label encoding).
- Scale or normalize numeric features for distance-based algorithms.
- Create interaction or ratio features when domain knowledge suggests they matter.
- Extract components from dates and text fields.

Chapter 6: Classification

Classification is a supervised learning task where the goal is to predict a discrete category or class label for each observation, based on input features.

Examples include spam detection (spam vs. not spam), churn prediction (churn vs. retain), and image recognition (identifying the object class in an image).

Common classification algorithms include logistic regression, decision trees, random forests, support vector machines, and gradient boosting.

Chapter 7: Regression

Regression is a supervised learning task where the goal is to predict a continuous numeric value, such as price, temperature, or sales volume.

Common regression algorithms include linear regression, decision tree regression, random forest regression, and gradient boosting regression.

Regression models are typically evaluated using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared.

Chapter 8: Model Evaluation

Model evaluation measures how well a trained model performs, using metrics appropriate to the task (classification or regression) and a held-out portion of data the model has not seen.

For classification: accuracy, precision, recall, F1-score, and ROC-AUC are commonly used. For regression: MAE, MSE, RMSE, and R-squared are commonly used.

No single metric tells the whole story — the right metric depends on the business problem and the cost of different types of errors.

Chapter 9: Precision and Recall

Precision and recall are two complementary metrics used to evaluate classification models, particularly when classes are imbalanced.

Precision answers: 'Of all instances predicted positive, how many were actually positive?' It is calculated as $\text{True Positives} / (\text{True Positives} + \text{False Positives})$.

Recall answers: 'Of all actual positive instances, how many did the model correctly identify?' It is calculated as $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$.

There is often a trade-off between precision and recall: increasing one can decrease the other. The F1-score is the harmonic mean of precision and recall, used to balance both.

Chapter 10: Overfitting and Underfitting

Overfitting occurs when a model learns the training data too well, including its noise and random fluctuations, resulting in excellent training performance but poor generalization to new data.

Underfitting occurs when a model is too simple to capture the underlying pattern in the data, resulting in poor performance on both training and new data.

Common techniques to reduce overfitting include cross-validation, regularization, pruning, gathering more training data, and simplifying the model. Underfitting is typically addressed by increasing model complexity or adding more relevant features.

Chapter 11: Train/Test Split

A train/test split divides a dataset into a training set (used to fit the model) and a test set (used to evaluate the model on unseen data), which helps estimate how well the model will generalize.

A common split ratio is 80% training and 20% testing, though this varies with dataset size. For model tuning, a separate validation set or cross-validation is often used alongside the test set.

It is important that the test set is never used during training or feature selection, to avoid data leakage and overly optimistic performance estimates.

Chapter 12: Common Data Science Mistakes

Even experienced practitioners can fall into avoidable traps that compromise the reliability of an analysis or model.

- Data leakage: information from the test set influencing training.
- Ignoring class imbalance when evaluating classification models.
- Drawing conclusions from correlation without considering causation.
- Not validating assumptions behind a statistical test or model.
- Skipping exploratory data analysis before modeling.
- Over-relying on a single metric without considering the business context.